Task 2: Hardware-Conscious Matrix Multiplication

CS683 Programming Assignment 1

Dhurandhar Programming

Advanced Computer Architecture
Autumn 2026

7 September 2026

Executive summary

This report evaluates software prefetching, SIMD vectorization, and their combination with cache tiling for single-precision matrix multiplication (SGEMM) in the "NT" layout used by `ggml/llama.cpp`, where $C[i][j] = \sum_p A[i][p]\,B[j][p]$ and the contraction dimension is contiguous in both operands. All three submitted kernels match the naive reference within the harness tolerance of 10^{-4} relative error, and the optimized kernel reproduces `llama.cpp`'s generated text token-for-token when injected into ggml. On the graded 1024^3 workload the official harness reported **43.6× speedup** for `matmul_optimized` and **100/100**; the best-of-rounds sweep in this report measured 45.9× at 1024^3 and 40.6× at 2048^3.

Three findings organise the analysis. First, **software prefetching alone does almost nothing for the scalar kernel** (0.95–1.05×): the naive loop is bound by the latency of its floating-point accumulation chain, and the hardware prefetchers already track its two unit-stride streams. Second, **SIMD is where the large gain comes from** (AVX2 retires 20.8× fewer instructions and runs 39–41× faster up to $n = 1024$), but the SIMD-only kernel becomes memory-bound and falls to 24× at $n = 2048$. Third, **cache tiling, not prefetching, repairs the memory-bound SIMD kernel**: tiling cuts L2 misses 3.7× at $n = 1024$ and restores 45×, while prefetching inside the vectorised inner loop is actively harmful (−22%) because its extra load-port micro-operations compete with the FMA stream. The only prefetch that pays is a sparse hint at the head of the next tile's rows, which is what the submitted kernels use.

Reference result at $M = N = K = 1024$ (official harness)

Stage	Median time (ms)	GFLOP/s	Speedup
`matmul_naive` (reference)	2818.3	0.76	1.00×
`matmul_simd` (AVX2, 4×3 register tile)	78.2	27.5	36.0×
`matmul_prefetch` (tiling + row-head prefetch)	63.9	33.6	44.1×
`matmul_optimized` (tiling + K-blocking + prefetch)	64.6	33.2	43.6×

At 1024^3 the working set still largely fits the 6 MiB L3 of the test machine, so the two blocked stages are within noise of each other. Their designs separate at $n \geq 1536$, where the optimized kernel keeps 40–44× while the prefetch stage drops to 32–35× and SIMD alone to 24× (Figure 9).

Experimental setup

Hardware and software

Item	Configuration
Processor	Intel Core i5-8265U (Whiskey Lake), 4 cores / 8 threads; reported maximum clock 1.6 GHz on this laptop
L1 data cache	32 KiB per core, 8-way, 64 B lines
L2 cache	256 KiB per core, 4-way
L3 cache	6 MiB shared, 12-way
SIMD support	SSE4.2, AVX2 + FMA; no AVX-512
Hardware prefetchers	L2 streamer, L2 adjacent-line, L1 DCU next-line, L1 DCU IP (MSR `0x1A4`)
Operating system	Ubuntu 22.04.5 LTS, Linux 6.8.0
Compiler	GCC 11.4.0
Submitted flags	`-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`
Profiling tool	`perf` 6.8.12

The assignment Makefile and flags were not modified. Every timing process was pinned to logical CPU 2 with `taskset -c 2`. Each configuration ran one warm-up call followed by five timed calls and reported the median; the naive kernel, which needs 25 s per call at $n = 2048$, was timed once at that size. Because this laptop's clock moves with temperature, every fast configuration was measured in three interleaved rounds and the **best median** is reported, which is the estimator

least affected by throttling. Speedup is

$$\text{speedup} = \frac{T_{\text{naive}}}{T_{\text{kernel}}} \quad \text{at the same } (M, N, K).$$

Absolute GFLOP/s are low because of the 1.6 GHz clock; conclusions rest on within-run speedups and counter ratios.

Hardware counters were collected with `perf stat` for `instructions`, `cycles`, `L1-dcache-load-misses`, `l2_rqsts.miss`, `LLC-load-misses` and the three `sw_prefetch_access` events (`t0`, `t1_t2`, `nta`). Each kernel was called enough times to dominate the process, a setup-only run (allocation and random fill) was subtracted, and the difference was divided by the number of calls, so every counter below is **per SGEMM call**.

Experimental kernels

The graded files are `src/matmul_simd.cpp`, `src/matmul_prefetch.cpp` and `src/matmul_optimized.cpp`. The report additionally uses ungraded variants in `experiments/kernels.cpp`, built with the same flags:

- `scalar_pf` (Task 2A): the naive loop plus `_mm_prefetch` on both operand rows, issued once per 64 B line at a configurable distance d (floats) and hint level.
- `sse128` (Task 2B): the same 4×3 register tile written with 128-bit SSE (`_mm_mul_ps` + `_mm_add_ps`), built with `-msse4.2`.
- `blocked_simd` (Task 2C): the AVX2 tile with M and N panel blocking and two switchable prefetch styles: *in-loop* (all seven operand streams, d floats ahead, once per line) and *row-head* (ℓ lines at the start of the next tile's rows). With both off it isolates "SIMD + tiling".

Correctness

The official harness passed its 256^3 smoke test and the graded run for all three stages. A separate harness of 51 shape-and-stride cases (M, N, K from 1 to 4100, including $M = 1$, odd sizes, and padded `lda`/`ldb`/`ldc`) passed for every kernel, with padding left untouched. The `make llama-demo` run confirmed that ggml invoked `matmul_optimized` and that the student build's output matched the stock build token-for-token on `stories260K`. Experimental variants had maximum relative error below 1.2×10^{-6}.

Kernel design

Register tile and loop structure

The inner product over K is a unit-stride dot product of an A row and a B row. The SIMD micro-kernel processes a 4×3 block of C: in each step of eight floats it loads three B vectors and, for each of four A rows, one A vector followed by three FMAs, so **seven loads feed twelve FMAs**. The twelve accumulators, three B vectors and one live A vector use exactly the sixteen AVX2 registers. The tile is written out by hand (loop unrolling): an equivalent version with an accumulator array under `-O2` measured about 10 GFLOP/s because the compiler kept the array in memory. After the K loop, each accumulator is reduced with a horizontal sum and any $K \bmod 8$ tail is added in scalar code; rows and columns that do not fill a tile use a vectorised single dot product.

Tile $M_R \times N_R$	ymm registers	FMAs per load	Time at 1024^3 (ms)	GFLOP/s
4×3	16	1.71	85	25.2
3×4	17	1.71	105	20.4
6×2	15	1.50	115	18.7
4×2	11	1.33	122	17.6
8×1	10	0.89	130	16.5
4×4	21	2.00	141	15.3
5×3	19	1.88	151	14.2

The sweep shows the register budget dominating: 4×4 and 5×3 have the best arithmetic intensity on paper but spill to the stack and finish behind 4×2. More A rows beat more B rows at equal area because only one A vector is live at a time. Unrolling the K loop by two changed nothing, so the FMA units, not loop overhead, are the limit.

The three submitted stages

- **`matmul_simd`** sweeps four A rows against every B row. The four A rows stay in L1, but every B row is re-read from L2/L3/DRAM for each group of four A rows, so traffic scales as $4MNK/4$ bytes.
- **`matmul_prefetch`** blocks M and N into panels of at most 96 KiB each so an A panel and a B panel co-reside in the 256 KiB L2, and prefetches two lines at the head of the next tile's A and B rows with `_MM_HINT_T0`. A new row is a jump to a new page that the hardware next-line prefetcher cannot anticipate; the sequential stream within a row is left to the hardware.
- **`matmul_optimized`** keeps a B panel of 192 KiB in L2 and sweeps the whole of M through it, so each B panel is loaded once from memory. K is additionally split into blocks of at most 1024 so that the four active A rows (16 KiB) stay inside the 32 KiB L1 at large K; the first K block stores its tile results and later blocks accumulate into C. (An earlier version zeroed C up front and always accumulated; the extra pass over C and the read-modify-write of C lines, which share the 4 KiB row stride and therefore the same cache sets as A and B, cost 6–9% at $n \leq 1024$.) The single prefetch site is the head of the next A row block.

Task 2A: software prefetching

Prefetch distance

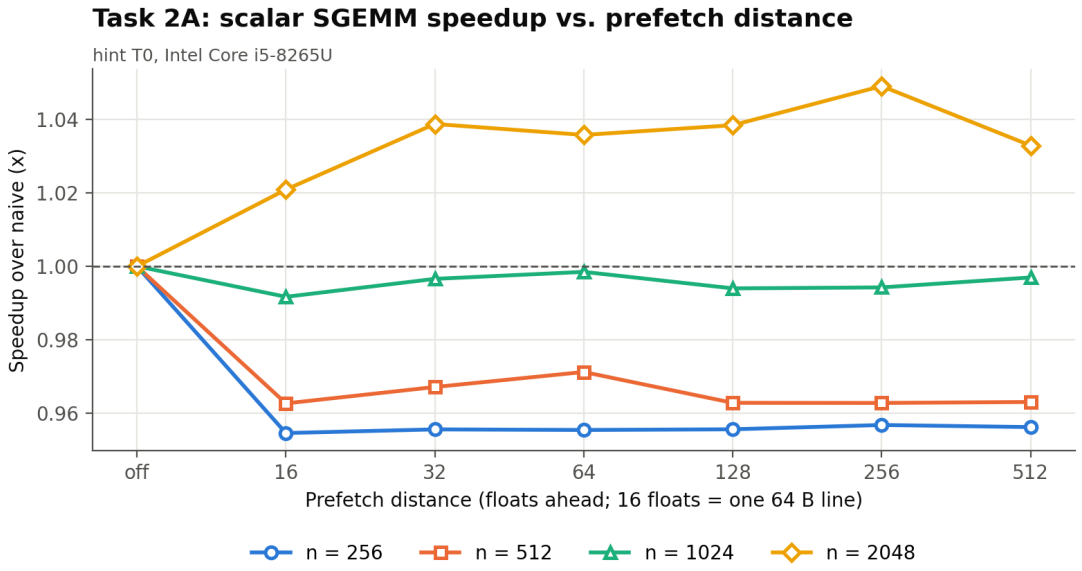


Figure 1: Scalar SGEMM speedup over naive versus prefetch distance (`_MM_HINT_T0`). Distance is in floats ahead of the current K index; 16 floats is one cache line.

For $n \leq 1024$ the prefetching scalar kernel is 0.95–$1.00\times$ the naive kernel at every distance; at $n = 2048$ it is 1.02–$1.05\times$, peaking at $d = 256$ floats (16 lines, $1.049\times$). Two counter measurements explain the flat curve. At $n = 1024$ the prefetch variant retires **9 835 M instructions against 5 403 M** for naive (+82%, from the per-element line test and the 135 M `prefetcht0` instructions), yet consumes **4 272 M cycles against 4 240 M** (+0.8%). The loop is bound by the four-cycle latency of the dependent `acc += a[p]*b[p]` chain, so the extra instructions execute in the shadow of that chain and the demand loads are already covered: L2 misses are identical (92.0 M with and without prefetch). Each 64 B line takes 16 elements $\times$ 4 cycles $= 64$ cycles to consume, so the hardware L2 streamer has ample time to stay ahead within a page. The small gain at $n = 2048$ appears when the B matrix (16 MiB) exceeds the L3: LLC demand misses fall from 7.95 M to 0.28 M because the software prefetch turns them into prefetch-side fills, and only then does a few percent of stall time become visible.

Cache fill level

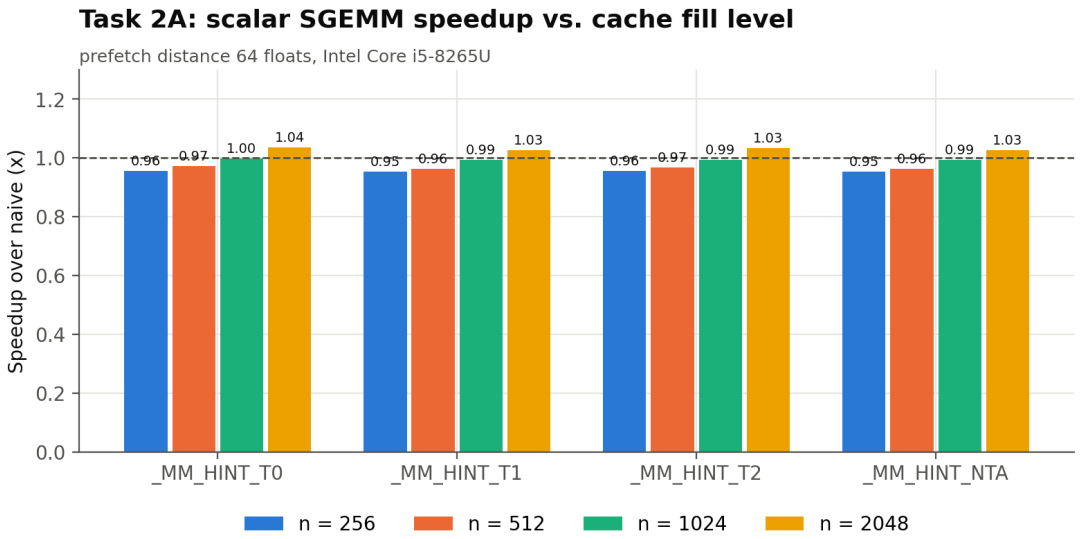


Figure 2: Scalar SGEMM speedup versus `_mm_prefetch` locality hint at $d = 64$ floats.

All four hints are within 1% of each other at every size (at $n = 2048$: T0 $1.036\times$, T2 $1.033\times$, T1 $1.027\times$, NTA $1.026\times$). The `sw_prefetch_access` counters confirm the encoding: T0 shows in `.t0`, T1 and T2 both in `.t1_t2`, NTA in `.nta`. The hints do change where the line lands: at $n = 1024$, L2 misses rise from 92 M (T0) to 106 M with T1/T2 and to **261 M with NTA**, because `prefetchnta` brings the line to L1 without filling L2, so the line is missed again in L2 when it is later evicted from the small L1. For this streaming access pattern the fill level is irrelevant to speed, and T0 is the safe default.

Matrix size and CPU metrics

Kernel	n	Instr. (M)	Cycles (M)	L1-D misses (M)	L2 misses (M)	LLC misses (M)	SW prefetch (M)
naive	256	84.9	61.1	1.06	0.82	0.001	0
scalar + pf d=64	256	154.2	63.8	1.06	0.80	0.000	2.18
naive	512	676.8	514.6	8.50	11.5	0.003	0
scalar + pf d=64	512	1230.9	527.6	8.51	11.6	0.001	17.1
naive	1024	5403.4	4239.5	68.0	92.0	0.163	0
scalar + pf d=64	1024	9834.6	4271.5	68.0	92.0	0.015	135.4
naive	2048	43157.0	34863.9	546.6	736.3	7.950	0
scalar + pf d=64	2048	78592.0	34396.4	546.6	736.6	0.276	1078.5

Misses grow with n^3 because every B row is streamed once per A row, but cycles per miss stay constant: the scalar kernel is not waiting for memory. Software prefetching redistributes misses between demand and prefetch counters without removing them.

Answers. (1) Speedup is flat at $\approx 1.0\times$ up to $n = 1024$ and reaches $1.05\times$ only at $n = 2048$, where the operands exceed the L3. (2) The best distance is 256 floats (16 lines) at $n = 2048$; for smaller sizes no distance beats the naive kernel. (3) The fill level does not change the speedup measurably; T0 was marginally best and NTA measurably increases L2 misses.

Task 2B: SIMD

The kernels use unaligned loads because `llama.cpp` passes arbitrary strides:

- 128-bit: `_mm_loadu_ps`, `_mm_mul_ps`, `_mm_add_ps`, `_mm_movehl_ps` / `_mm_shuffle_ps` for the horizontal sum;
- 256-bit: `_mm256_loadu_ps`, `_mm256_fmadd_ps`, `_mm256_extractf128_ps` + SSE adds for the horizontal sum.

The FMA intrinsic was chosen because the dot product is a multiply-accumulate; one instruction replaces a multiply and a dependent add, halves the instruction count of the inner loop and removes one rounding step. The submitted kernel is 256-bit AVX2 because that is the pinned build target. The 128-bit build is an analysis-only variant compiled with `-msse4.2`; this processor has no AVX-512, so the 512-bit point could not be measured.

Retired instructions and cycles at $n = 1024$

Version	Instructions (M)	Reduction	Cycles (M)	IPC
Scalar (naive)	5403.4	1.00×	4239.5	1.27
128-bit SSE	977.9	5.53×	284.3	3.44
256-bit AVX2 + FMA	259.2	20.8×	98.6	2.63

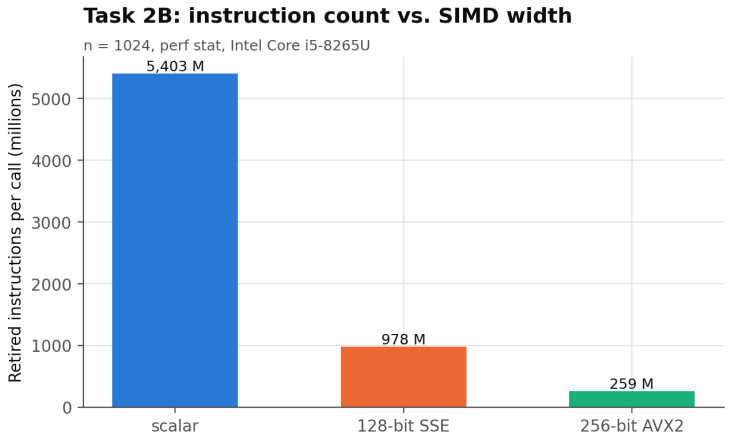


Figure 3: Retired instructions per SGEMM call at $n = 1024$ for the three register widths.

The instruction reduction exceeds the vector width because the register tile also removes loads: the scalar loop performs two loads per multiply-add, whereas the 4×3 AVX2 tile performs seven loads per twelve 8-wide FMAs. Cycles fall by $43\times$, more than the instruction reduction, because the scalar loop's IPC is limited by its dependent accumulation chain while the tile keeps twelve independent chains in flight.

SIMD width and matrix size

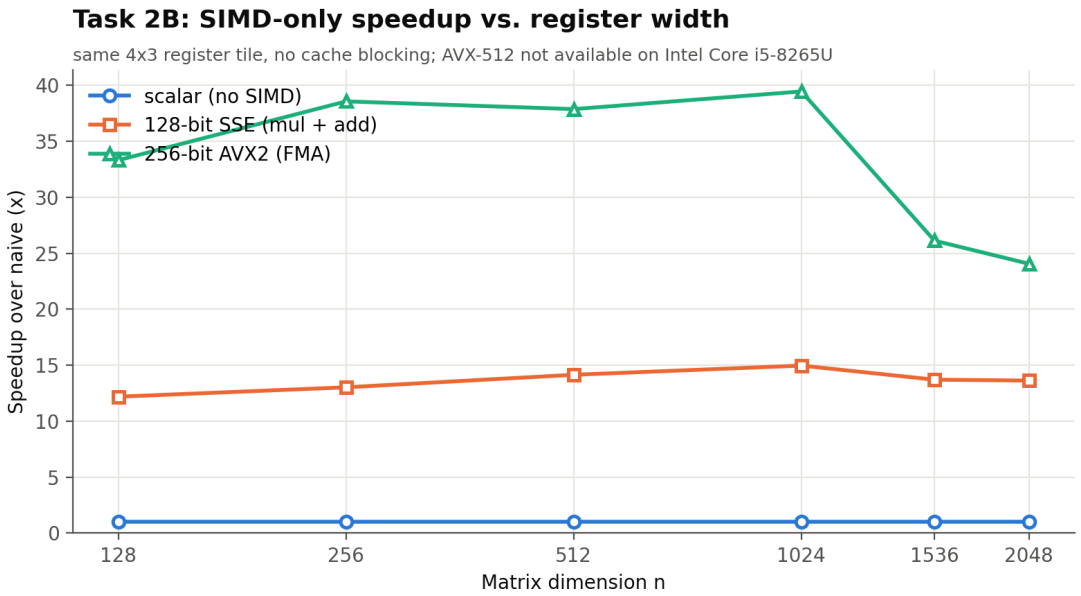


Figure 4: SIMD-only speedup over naive for 128-bit and 256-bit registers, same 4×3 tile, no cache blocking.

SSE holds 12–$15\times$ at every size. AVX2 holds 33–$41\times$ up to $n = 1024$ and then drops to $26\times$ at $n = 1536$ and $24\times$ at $n = 2048$. The counters locate the cliff: at $n = 2048$ the AVX2 kernel's LLC misses reach 4.69 M per call and its L2 misses 225 M, against 116 M for the tiled kernel. Once A (16 MiB) and B no longer fit the 6 MiB L3, the un-blocked sweep re-reads every B row from DRAM for every group of four A rows; the arithmetic is fast enough that memory becomes the bottleneck. The SSE kernel is unaffected because it computes slowly enough to hide the same traffic.

Answers. (1) Speedup rises with register width at all sizes, but the AVX2 advantage shrinks from $2.7\times$ over SSE at $n \leq 1024$ to $1.8\times$ at $n = 2048$ as the kernel becomes memory-bound. (2) 256-bit AVX2 gives the maximum speedup at every size; a 512-bit variant could not be tested on this processor and would be expected to help only within the cache-resident regime.

Task 2C: software prefetching + SIMD

Cache tiling is the necessary companion: with SIMD the kernel is memory-bound, so both tiling and prefetching are applied to the AVX2 kernel and the same sweeps as Task 2A are repeated.

Prefetch distance

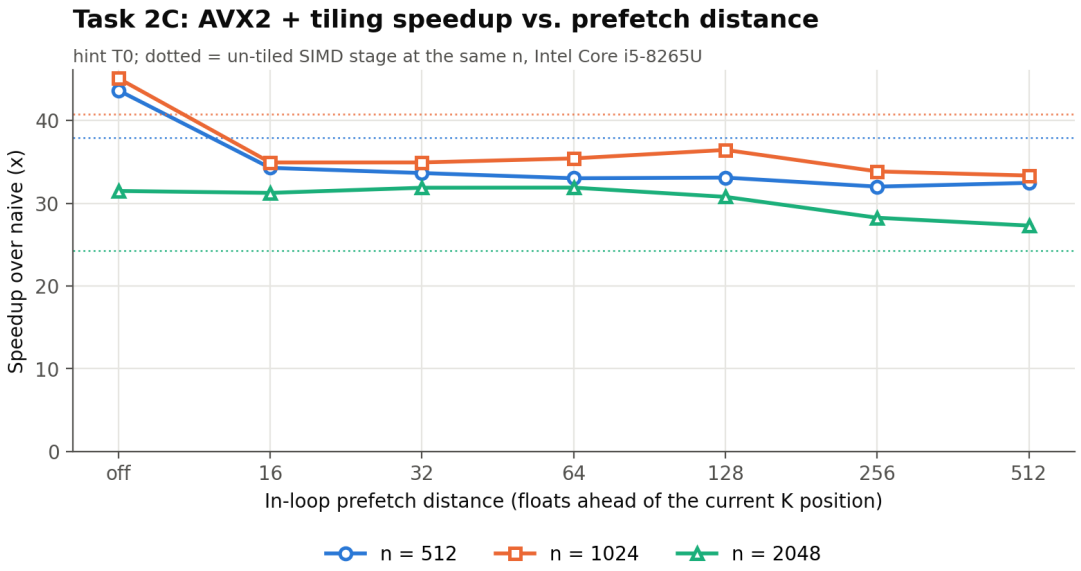


Figure 5: AVX2 + tiling speedup over naive versus in-loop prefetch distance (T0). Dotted lines are the un-tiled SIMD stage.

Tiling alone lifts $n = 1024$ from $40.7\times$ to $45.1\times$ and $n = 2048$ from $24.3\times$ to $31.5\times$. Prefetching inside the vectorised K loop then **lowers** the speedup at every distance for $n \le 1024$ (to 33–$36\times$, about -22%) and is at best neutral at $n = 2048$ ($31.9\times$ at $d = 64$ versus $31.5\times$ without). The counters show why: at $n = 1024$ the in-loop variant retires 401 M instructions instead of 263 M and needs 115 M cycles instead of 90 M, while its L2 misses are already as low as the tiled kernel's (6.5 M versus 7.0 M). The inner loop issues seven loads per twelve FMAs; adding seven prefetch micro-operations per line makes the load ports the bottleneck instead of the FMA units, and there is nothing left to hide because the hardware prefetcher already streams within each row. Increasing the distance beyond 128 floats makes things worse at $n = 2048$, since prefetching half a kilobyte ahead on seven streams pushes lines out of the 32 KiB L1 before they are used.

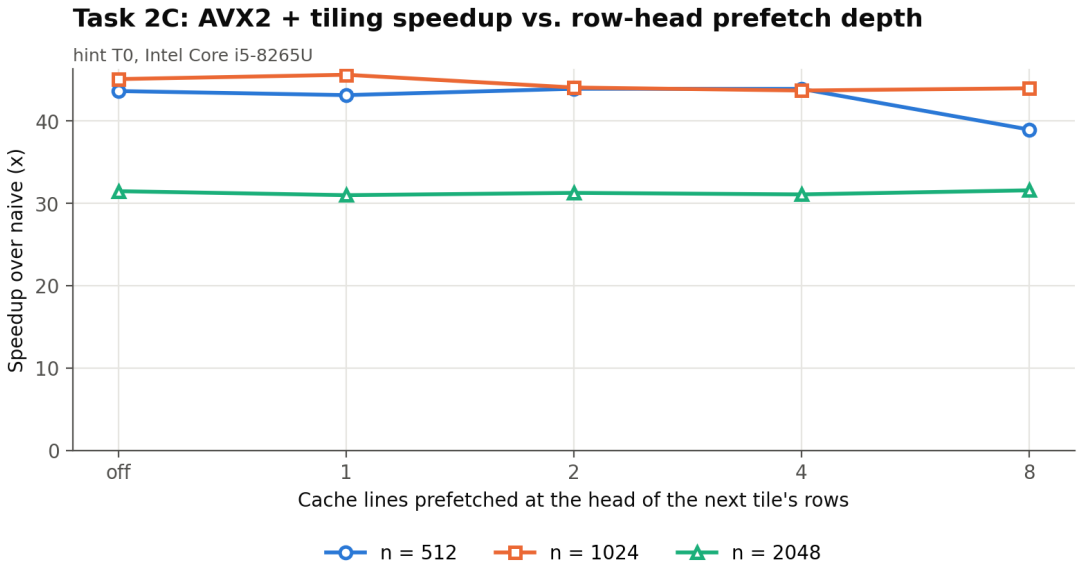


Figure 6: AVX2 + tiling speedup versus the number of lines prefetched at the head of the next tile's rows (T0).

The sparse row-head prefetch behaves differently. One or two lines per upcoming row cost only 0.5 M prefetch instructions at $n = 1024$ (+1.7% instructions, +0.7% cycles) and keep the tiled kernel at 44–47×; eight lines start to cost measurable time at $n = 512$. This is the form of prefetching the submitted `matmul_prefetch` uses (two lines) and `matmul_optimized` uses (one line per A row).

Cache fill level

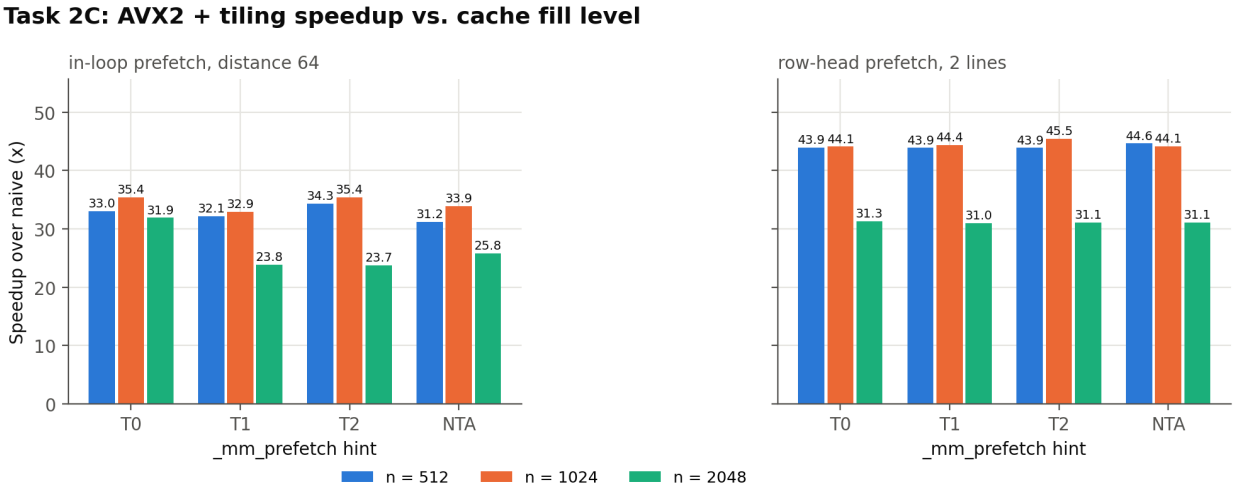


Figure 7: AVX2 + tiling speedup versus hint for in-loop prefetch at $d = 64$ (left) and row-head prefetch of two lines (right).

For row-head prefetch all hints are equivalent (43–47×; the 47.5× of NTA with one line at $n = 1024$ is within the round-to-round spread). For in-loop prefetch at $n = 2048$ the hint matters: T0 keeps 31.9×, whereas T1, T2 and NTA fall to 23.7–25.8×. The counters at $n = 1024$ show T1/T2 raising L2 misses from 6.5 M to 8.0–8.4 M and NTA to 19.0 M: a line prefetched into L2 only, or bypassing L2, is fetched a second time by the demand load, so on this machine the only hint that does not add traffic is T0.

Matrix size and CPU metrics

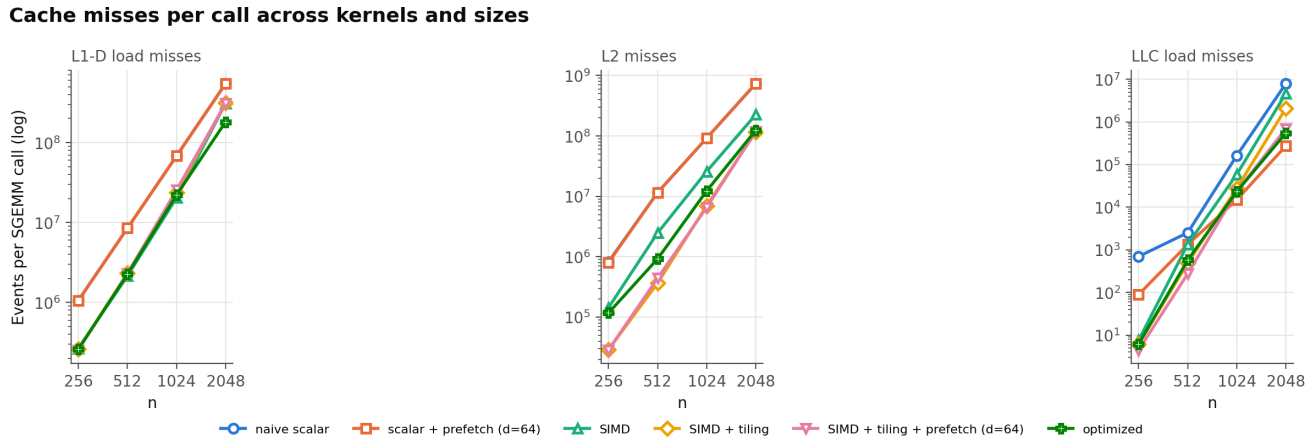


Figure 8: Cache misses per SGEMM call (log scale) for scalar, SIMD, tiled, prefetched and optimized kernels.

Kernel at $n = 1024$ / 2048	Instr. (M)	Cycles (M)	L1-D miss (M)	L2 miss (M)	LLC miss (M)	Speedup
SIMD	259 / 2055	98.6 / 1206	20.4 / 310	25.8 / 225	0.06 / 4.69	39.5 / 24.1
SIMD + tiling	263 / 2072	89.9 / 1017	23.7 / 313	7.0 / 116	0.03 / 2.10	44.6 / 31.1
SIMD + tiling + in-loop pf d=64	401 / 3183	114.7 / 1055	25.6 / 309	6.5 / 119	0.02 / 0.70	35.4 / 31.9
SIMD + tiling + row-head pf (2 lines)	268 / 2090	90.5 / 1020	24.1 / 317	7.1 / 116	0.02 / 1.75	44.1 / 31.3
optimized (B panel + K blocks)	260 / 2105	90.0 / 816	22.1 / 181	12.3 / 124	0.02 / 0.55	45.9 / 40.6

Tiling divides L2 misses by 3.7 at $n = 1024$ and halves LLC misses at $n = 2048$. The optimized kernel's K-blocking shows up in the L1 column at $n = 2048$: 181 M misses instead of 313 M, because a four-row A block of $K = 1024$ floats is 16 KiB and stays resident while the B panel streams past, whereas the full $K = 2048$ rows need the entire 32 KiB L1. That is the difference between $31\times$ and $40\times$ at $n = 2048$.

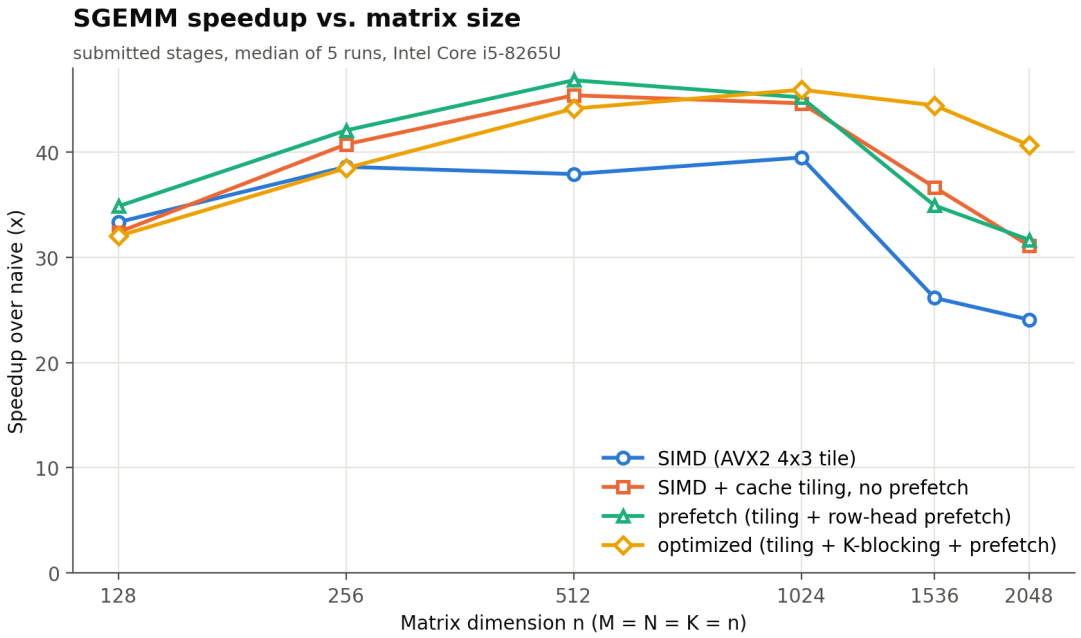


Figure 9: Speedup of the submitted stages over naive versus matrix size (README plot 1).

Hardware prefetchers

The four core prefetchers (L2 streamer, L2 adjacent-line, L1 next-line, L1 IP-stride) were disabled together by writing `0xF` to MSR `0x1A4` on every core, the subset of kernels below was re-timed, and the register was restored. "On" timings are the best over the study's own round and the earlier best-of-rounds sweeps of the same configurations.

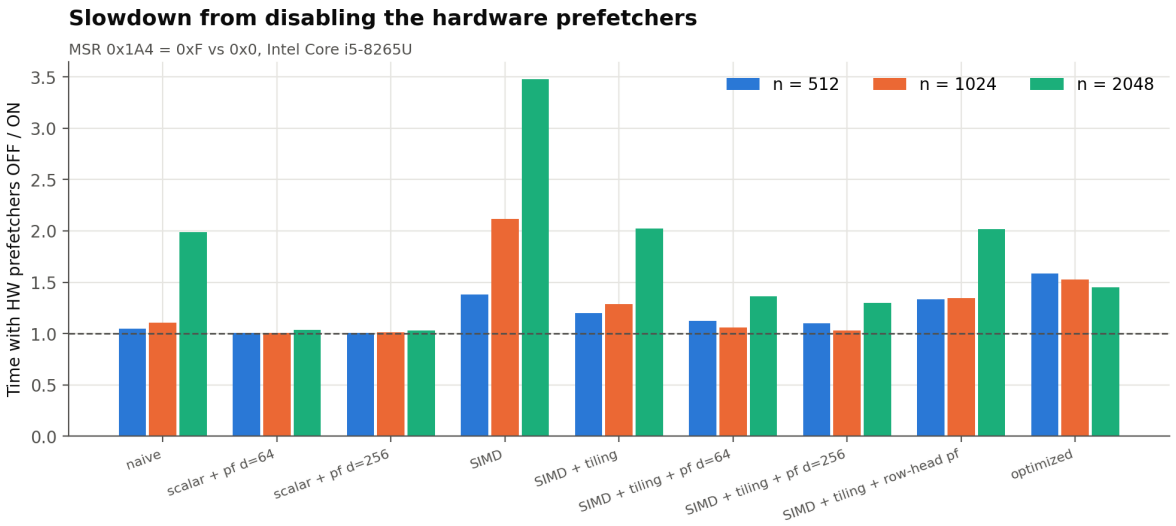


Figure 10: Slowdown of each kernel when all hardware prefetchers are disabled.

Kernel	$n = 512$ (ms)		$n = 1024$ (ms)		$n = 2048$ (ms)	
	on	off	on	off	on	off
naive	337	354	2829	3123	23530	46742
scalar + pf d=64	347	350	2831	2856	22718	23487
scalar + pf d=256	350	353	2848	2877	22430	23116
SIMD	9.0	12.5	69.4	147.1	967	3363
SIMD + tiling	7.5	9.0	62.7	80.8	750	1520
SIMD + tiling + in-loop pf d=64	9.9	11.2	79.9	84.5	734	999
SIMD + tiling + in-loop pf d=256	10.0	11.1	83.4	85.8	834	1083
SIMD + tiling + row-head pf (2 lines)	7.7	10.3	63.9	85.8	755	1522
optimized	8.1	12.3	65.8	94.0	583	843

Disabling the hardware prefetchers exposes how much they were doing. The naive kernel slows by $1.10\times$ at $n = 1024$ and by $1.99\times$ **at** $n = 2048$: once B streams from DRAM, its per-line demand miss (≈300 cycles) exceeds the 64 cycles of arithmetic per line. The un-blocked SIMD kernel is hit hardest, $2.1\times$ at $n = 1024$ and $\mathbf{3.5}\times$ at $n = 2048$, because it has the highest bandwidth demand per instruction. Cache tiling reduces the dependence to $2.0\times$, and the optimized kernel's K-blocking to $1.45\times$, because both replace DRAM streams with L1/L2 reuse. At $n = 1024$ the counters agree: naive LLC demand misses rise from 0.15 M to 7.26 M per call without the prefetchers, SIMD from 0.06 M to 1.14 M, and SIMD + tiling only to 0.50 M.

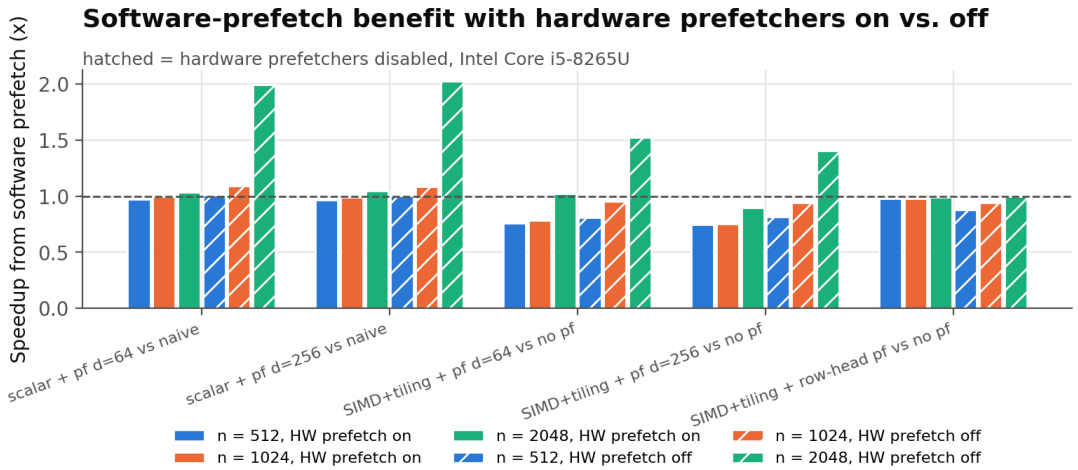


Figure 11: Speedup obtained from software prefetch with hardware prefetchers on (solid) and off (hatched).

With the hardware prefetchers off, software prefetching finally has something to do. In the scalar kernel at $n = 2048$ it recovers essentially the whole loss: $d = 64$ and $d = 256$ run in 23.5 s and 23.1 s against 46.7 s for naive, a $2.0\times$ **gain** where the same hint gave $1.04\times$ with the prefetchers on, and LLC demand misses at $n = 1024$ drop from 7.26 M back to 0.03 M. In the vectorised kernel the in-loop prefetch at $d = 64$ cuts the tiled kernel's time from 1520 ms to 999 ms ($1.52\times$) and its LLC misses from 0.50 M to 0.03 M, while $d = 256$ helps less ($1.40\times$) because seven streams prefetched a kilobyte ahead overflow the 32 KiB L1. The row-head prefetch, which only covers the first lines of each row, provides no recovery (1522 ms) because the rest of every row now misses. This confirms the earlier interpretation: on this machine the in-loop software prefetch is redundant with the L2 streamer and L1 next-line prefetchers, and its cost is visible only because they are doing the job already. Software prefetching is a substitute for hardware prefetching rather than a complement, and the hardware version is cheaper because it consumes no issue slots.

Final comparison

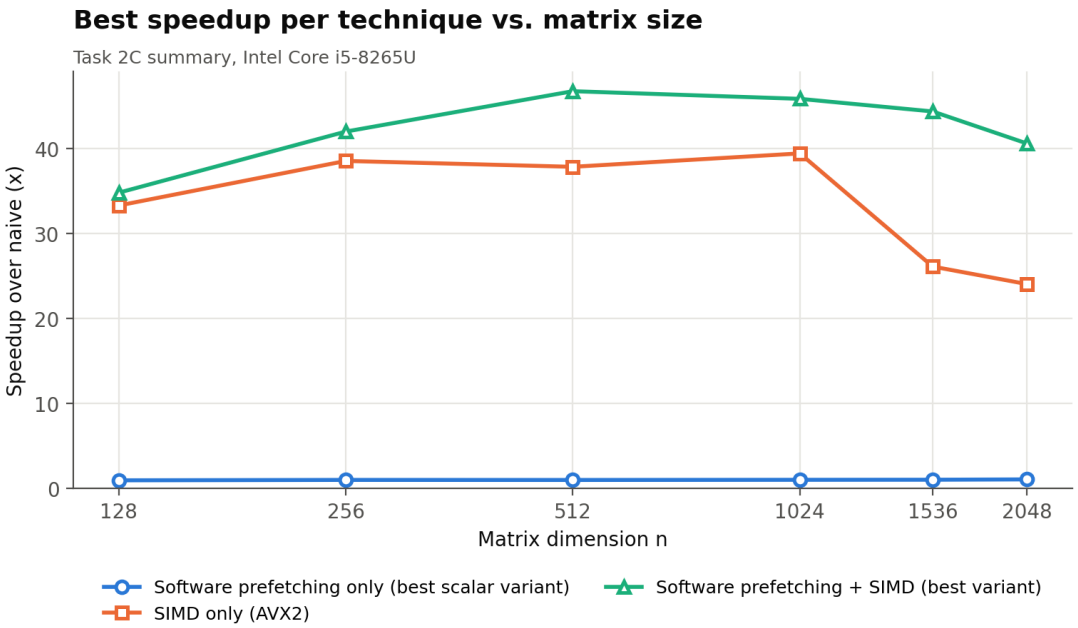


Figure 12: Best speedup achieved by each technique versus matrix size.

Technique (best variant)	n=128	256	512	1024	1536	2048
Software prefetching only (scalar)	0.94×	0.99×	0.98×	1.01×	1.01×	1.05×
SIMD only (AVX2)	33.3×	38.6×	37.9×	39.5×	26.1×	24.1×
Software prefetching + SIMD (+ tiling)	34.8×	42.0×	46.8×	47.5×	44.4×	40.6×

Answers. (1) With SIMD, speedup grows with size up to $n = 1024$ as fixed costs amortise, then the un-blocked kernel collapses beyond the L3 while the blocked kernels retain $40\times$. (2) The best in-loop distance is "none"; the best useful prefetch is one to two lines at the head of the next rows. (3) T0 gives the maximum speedup whenever prefetching is on the critical path; for the row-head hint any level works.

Conclusions

1. The scalar SGEMM loop is latency-bound, not memory-bound: software prefetching changes the instruction count by 82% and the cycle count by under 1%, and only at $n = 2048$ does it gain 5%. Prefetch distance and hint level are irrelevant in that regime, and NTA measurably increases L2 traffic.
2. Explicit AVX2 with a 4×3 register tile is the decisive optimization: $20.8\times$ fewer instructions and 39–$41\times$ speedup up to $n = 1024$. The tile shape is dictated by the sixteen-register file; wider tiles spill and lose.
3. SIMD turns the kernel memory-bound. At $n = 2048$ its LLC misses grow to 4.7 M per call and speedup falls to $24\times$. Cache tiling, which cuts L2 misses $3.7\times$, is what recovers it, and K-blocking keeps the A block inside L1 for another 30% at $n = 2048$.
4. Software prefetching in the vectorised inner loop is harmful (-22% at $n \leq 1024$) because it converts an FMA-bound loop into a load-port-bound one while the hardware prefetcher already covers the streams. The only prefetch that helps is a sparse hint at row boundaries, which the hardware cannot predict, and that is what the submitted kernels do.
5. The submitted `matmul_optimized` reaches $43.6\times$ in the graded harness, $45.9\times$ at 1024^3 in the controlled sweep and $40.6\times$ at 2048^3, computes correct results for arbitrary shapes and strides, and reproduces `llama.cpp`'s output exactly.

The submitted code modifies only the three permitted source files and preserves the supplied build system and compiler flags; all experimental variants live in `experiments/` and were used only to produce this report.